Lab 2: git and GitHub, step by step

You get a small crawler. You put it under git, push it to GitHub, keep its secret out of the repo, and work with branches, a merge and a pull request. Do each step together with the class and raise your hand when you reach the checkpoint.

All commands run in a terminal opened in the `book-crawler` folder.

Step 0. Before we start

Set your name once (use any email, it appears in your commits):

```
git config --global user.name "Your Name"
git config --global user.email "you@example.com"
```

Unzip `lab2.zip`. It contains the `book-crawler` folder and this guide as a PDF. Open a terminal inside the `book-crawler` folder (not the folder above it). You also downloaded `quotes.py`: **leave it where it is** until your teacher says so (step 6).

Step 1. Put it under git and push it

```
git init
git add .
git commit -m "initial commit"
git branch -M main
```

On github.com click **New repository**, give it a name, and **do not tick** README, .gitignore or license. Then copy its URL:

```
git remote add origin https://github.com/<your-user>/<your-repo>.git
git push -u origin main
```

Checkpoint: refresh the GitHub page, you see your files. `git log --oneline` shows one commit.

Now create the environment and install the libraries (it runs while we talk):

```
python -m venv .venv
Windows PowerShell:  .venv\Scripts\Activate.ps1
Windows cmd:         .venv\Scripts\activate.bat
Mac / Linux:         source .venv/bin/activate
pip install requests beautifulsoup4 python-dotenv
```

Step 2. Keep secrets and junk out of git

1. Run `python crawler.py`. It works, even with the empty `API_KEY`.

2. Type a fake key into the empty line of `crawler.py` and run it again:

```
API_KEY = "sk-demo-1234567890abcdef"
```

   It works. But the key is now written in the code. Who can see it if the repo is shared?
3. Move it to `.env`, which holds secrets as `NAME=value` pairs. Write this in `.env`:

```
API_KEY=sk-demo-1234567890abcdef
```

4. Run `git status`. `.env` is listed as modified. Why? It was in your first commit (empty), so git is tracking it, and committing now would publish the key. Stop tracking it but keep the file:

```
git rm --cached .env
```

5. Make the code read the key from `.env`. In `crawler.py` add `import os` and `from dotenv import load_dotenv` at the top, then replace the key line with:

```
load_dotenv()
API_KEY = os.getenv("API_KEY")
```

   Run the crawler again, it must still work.
6. Run `git status`. `.env`, `.venv/` and `books.jsonl` are untracked. `git add .` would add them all, including `.env`. Tell git to leave them alone: create `.gitignore` with these lines:

```
.env
.venv/
__pycache__/
books.jsonl
```

   Run `git status` again, they are gone from the list. (If `.venv` is still listed because it was committed, use `git rm -r --cached .venv`.)
7. Write `requirements.txt` with one library per line: `requests`, `beautifulsoup4`, `python-dotenv`. Compare with `pip freeze`. In step 1 you installed the libraries one by one. With this file, anyone can install all of them with one command, and you can run it too:

```
pip install -r requirements.txt
```

   Each line says `Requirement already satisfied`, because you installed them earlier. On a new computer, or in a new venv, the same command installs everything in one go.
8. Check, then commit. **Do not push yet**, the next step pushes once:

```
git status
git add .
git status
git commit -m "load key from .env, add .gitignore and requirements"
```

Checkpoint: `git ls-files` lists `.gitignore`, `README.md`, `crawler.py` and `requirements.txt`, and no `.env`.

Step 3. Change something and commit

In `crawler.py` change `PAGES = 2` to `PAGES = 3`.

```
git diff
git add crawler.py
git commit -m "crawl 3 pages"
git push
git log --oneline
```

This push sends both of your new commits (the one from step 2 and this one).

Checkpoint: `git diff` showed one changed line, `git log` has 3 commits, and on GitHub there is no `.env`.

Step 4. A branch

```
git branch new-feature
git switch new-feature
```

Change the line that adds a book to also store the rating:

```
        books.append({"title": item.h3.a["title"], "price":
item.select_one(".price_color").text, "rating": item.select_one(".star-rating")["class"]
[1]})
```

Run the crawler and look at `books.jsonl`: each book has a rating. Then:

```
git add crawler.py
git commit -m "add rating"
git push -u origin new-feature
```

Checkpoint: `git branch` shows `* new-feature`, and GitHub lists two branches.

Step 5. Merge it into main

```
git switch main
```

Open `crawler.py`: the rating is gone. It lives only on the branch. Now bring it in:

```
git merge new-feature
git log --oneline
git push
```

Checkpoint: the rating is back in `crawler.py` and `main` and `new-feature` end at the same commit.

Step 6. A pull request

Now copy `quotes.py` into your repo folder.

```
git branch quotes
git switch quotes
python quotes.py
git status
git add quotes.py
git commit -m "add quotes crawler"
git push -u origin quotes
```

(We add the file by name, because running `quotes.py` created `quotes.jsonl`, which we do not want to commit.)

On GitHub:

1. Click **Compare & pull request**.
2. Write a title and a short description (what you changed and why).
3. Open the **Files changed** tab and read the diff.
4. Scroll to the checks at the bottom of the page. A test called `tests` runs on a clean computer and, after about 30 seconds, shows a green tick. Click **Details** to see what it did.
5. Click **Merge pull request**, then **Confirm merge**.

Back in the terminal, your laptop does not know about the merge yet:

```
git switch main
git pull
git log --oneline --graph
```

Checkpoint: the check showed a green tick, `quotes.py` is on `main`, and the graph shows the branch splitting off and joining back.

Extra steps (if you finish early)

Do them in any order.

1. **Fresh clone test.** Clone your repo into a new folder, create a new venv, run `pip install -r requirements.txt`, create a `.env` with the key, and run the crawler. This is what a teammate would do. What did they have to guess?

2. **Look inside commits.** `git log -p`, `git show <id>`. Find your hashes: what do the first 7 characters stand for?
3. **Undo a change.** Edit a file and run `git restore <file>` to throw the change away. Then make a bad commit and undo it with `git revert <id>`.
4. **A conflict, on purpose.** Make two branches that change the same line of `crawler.py` differently, merge both into `main`, and resolve the markers that git leaves in the file (`<<<<<<<`, `=======`, `>>>>>>>`).
5. **Write a README.** Add a section to `README.md`: what the project does, how to set up the environment, how to run it.
6. **A test of your own.** Write a small function that turns a price like `£51.77` into the number `51.77`, and a second test with one `assert` for it in a new test file. Run `pip install pytest` and then `pytest`. Break the assert on purpose and see what a failing test looks like.
7. **A `.env.example`.** Your `.env` stays out of git, so a teammate who clones the repo does not know which settings to provide. Create `.env.example` with the variable names and a fake value (`API_KEY=your-key-here`) and commit it. Redo the fresh clone test with it.
8. **Another PR.** Change the number of pages through a variable in `.env`, open a PR for it, and describe it so that someone else can review it.